

Project 4: Automated Banking Website using Agile

1. Abstract

The Automated Banking Website project involves the creation of a secure digital banking platform that allows customers to manage their finances online. Developing this under the **Agile methodology** ensures that critical security features and transaction modules are prioritized in the "Product Backlog" and delivered through iterative Sprints. The core focus of this project is the integration of **Selenium Automation** to verify complex banking workflows, such as fund transfers and statement generation, ensuring high reliability and data integrity.

+2

2. Introduction

- **2.1 Introduction:** This project focuses on building a modern web-based banking application that provides 24/7 financial services to users.
- **2.2 Problem Identification:** Traditional manual banking is time-consuming, and manual testing of web banking is prone to human error and security vulnerabilities.
- **2.3 Need of the project:** There is a critical need for a secure, automated, and efficient banking platform that reduces operational costs and improves user accessibility.
- **2.4 Project Scheduling:** The development is divided into four Sprints: Sprint 1 (Account Creation & Security), Sprint 2 (Fund Transfers), Sprint 3 (Automated Statement Generation), and Sprint 4 (Regression Automation Testing).
- **2.5 Objectives:** To provide a secure environment for transactions, implement real-time balance updates, and automate the testing of core banking features.

3. Software Requirement Specification

- **3.1 Purpose:** To offer a reliable and automated digital banking experience for retail customers.

- **3.2 Scope:** Includes login authentication, balance enquiry, internal/external fund transfers, and transaction history.
 - **3.3 Hardware/Software Requirement (minimum):**
 - **Hardware:** Core i5 Processor, 8GB RAM, 100GB Disk Space.
 - **Software:** Java JDK 17, MySQL Database, Apache Tomcat, Selenium WebDriver.
 - **3.4 Tools:** Eclipse IDE, Maven, Jira, and TestNG for automation framework management.
 - **3.5 Software Process Model: Agile Scrum Model,** allowing for continuous integration and rapid response to security updates.
-

4. System Design

- **4.1 Data Dictionary:** Defines critical fields like Account_No, Transaction_ID, IFSC_Code, and Current_Balance.
 - **4.2 ER Diagram:** Illustrates the relationship between Customer, Account, Transaction, and Branch.
 - **4.3 DFD (Data Flow Diagram):** Shows the secure flow of data during a fund transfer request, from user input to bank server validation.
 - **4.4 Activity Diagram:** Outlines the logic for multi-factor authentication and transaction authorization.
-

5. Implementation

- **5.1 Program Code:** Implemented using Java for the backend logic and Selenium WebDriver for automating the testing of login security and transfer flows.
 - **5.2 Output Screens:** Snapshots of the Account Summary Dashboard, Secure Transfer Form, and E-Statement View.
-

6. Testing

- **6.1 Test Data:** Use of valid/invalid account numbers, expired OTPs, and boundary transaction limits.
 - **6.2 Test Result:** Automated test logs showing a 100% pass rate for the "Zero Balance Transfer Prevention" and "SQL Injection Vulnerability" checks.
-

7. User Manual

- **7.1 How to use project guidelines:** Instructions on registering for net banking, setting up transaction passwords, and downloading receipts.
 - **7.2 Screen Layouts and Description:** Guide explaining the "Lock Account" feature, "Mini Statement" sidebar, and "Contact Support" tools.
-

8. Project Applications and Limitations

- **Applications:** Suitable for retail banks, credit unions, and microfinance institutions.
 - **Limitations:** Heavily dependent on network stability for real-time OTP delivery; requires high-end server-side encryption.
-

9. Conclusion and Future Enhancement

- **Conclusion:** The Agile approach, combined with Selenium automation, resulted in a highly secure and functional banking platform.
 - **Future Enhancement:** Integration of Biometric authentication (Face/Fingerprint) and Blockchain-based transaction ledgers for enhanced security.
-

10. Bibliography & References

- Referencing Selenium HQ, Oracle Java Documentation, and Agile Alliance guidelines.